

Reporting API

e-Invoicing Sandbox Release (2.1.0)

OAS 3.0.1

ZATCA wants to provide Taxpayers and Developers of Taxpayer e-invoicing solutions and devices the opportunity to test the integration of the systems with a ZATCA Sandbox environment prior to the launch of the production system. The Integration Sandbox (ISB) should enable solution developers to simulate the integration calls/requests that will be required later as part of the registration process and the submission of e-invoices, credit and debit notes to the production system. The Sandbox backend will accordingly simulate the validations and responses as part of the Cryptographic Stamp Identifiers issuance, renewal and revocation as well as the Reporting and Clearance function.

Although the ISB will give ZATCA an indication of the adoption rate for e-invoicing solutions in the market, it will not be mandatory to complete Sandbox testing as a pre-requisite for Registration/Taxpayer onboarding or accessing the production system. Similar to the Compliance and Enablement Toolbox (CET), the ISB is also aimed at Developers to build/update their solutions which are in line with ZATCA specifications and standards and are able to integrate with a ZATCA backend. Accordingly access to the ISB test/mock APIs will not be limited to Taxpayers and any user can register for a Developer account to access the ISB test/mock APIs and associated documentation. This registration will enable ZATCA to monitor the solution providers who intent to develop/update their solutions to integrate with ZATCA.

It should be noted that although the ISB will simulate most of the core functionalities of the production system, any validations that require integrations/access with external systems and/or storage as well as scenarios involving any backend exceptional handling (for example overriding the clearance process) will not be part of the ISB and will be covered by the core solution. Accordingly the ISB should not be considered as representative of all integrations and/or APIs that will be part of the production system.

Kindly note that validations which can result in an UBL XSD error also apply to optional fields if the tag is present and data input is not compliant. This includes leaving such fields blank. However if the tag itself is absent than the validations will not be performed.

This swagger documents the set of apis for the Sandbox (ISB) solution.

Developers can also refer to section 2.3.10 of the Developer Portal User Manual for additional guidance and steps.

Servers

<https://gw-fatoora.zatca.gov.sa/e-invoicing/developer-portal>

Reporting Model Endpoint(s)

[illegible]

Responses

Code	Description	Links
------	-------------	-------

200 HTTP OK. Returned on successful validation of simplified invoice.

Media type

application/json

Example Value | **Reported**

```
{
  "validationResults": {
    "infoMessages": [
      {
        "type": "INFO",
        "code": "XSD_ZATCA_VALID",
        "category": "XSD_validation",
        "message": "Complied with UBL 2.1 standards in line with ZATCA specifications",
        "status": "PASS"
      }
    ],
    "warningMessages": [],
    "errorMessages": [],
    "status": "PASS",
    "reportingStatus": "REPORTED"
  }
}
```

202 HTTP Accepted. Returned when the invoice is reported with warnings

Media type

application/json

Example Value | **Reported**

```
{
  "validationResults": {
    "infoMessages": [
      {
        "type": "INFO",
        "code": "XSD_ZATCA_VALID",
        "category": "XSD_validation",
        "message": "Complied with UBL 2.1 standards in line with ZATCA specifications",
        "status": "PASS"
      }
    ],
    "warningMessages": [
      {
        "type": "WARNING",
        "code": "BR-KSA-98",
        "category": "KSA",
        "message": "[BR-KSA-98] - The simplified invoice should be submitted within 24 hours of issuing the invoice.",
        "status": "WARNING"
      }
    ],
    "errorMessages": [],
    "status": "WARNING",
    "reportingStatus": "REPORTED"
  }
}
```


400 HTTP Bad Request. Returned when the submitted request is invalid.

Media type

application/json

Example Value | Invalid Invoice Hash

```
{
  "validationResults": {
    "infoMessages": [
      {
        "type": "INFO",
        "code": "XSD_ZATCA_VALID",
        "category": "XSD validation",
        "message": "Complied with UBL 2.1 standards in line with ZATCA specifications",
        "status": "PASS"
      }
    ],
    "warningMessages": [],
    "errorMessages": [
      {
        "type": "ERROR",
        "code": "invalid-invoice-hash",
        "category": "INVOICE_HASHING_ERRORS",
        "message": "The invoice hash API body does not match the (calculated) Hash of the XML",
        "status": "ERROR"
      },
      {
        "type": "ERROR",
        "code": "invoiceHash_QR_CODE_INVALID",
        "category": "QR_CODE_VALIDATION",
        "message": "Invoice xml hash does not match with qr code invoice xml hash",
        "status": "ERROR"
      }
    ],
    "status": "ERROR",
    "reportingStatus": "NOT_REPORTED"
  }
}
```

Example Value | Missing Invoice Hash

```
{
  "validationResults": {
    "infoMessages": [],
    "warningMessages": [],
    "errorMessages": [
      {
        "type": "ERROR",
        "code": "InvoiceHash-Errors",
        "category": "Invalid-InvoiceHash",
        "message": "Document hash is not present in the API body",
        "status": "ERROR"
      }
    ],
    "status": "ERROR",
    "reportingStatus": "NOT_REPORTED"
  }
}
```

Example Value | Missing Invoice

```
{
  "validationResults": {
    "infoMessages": [],
    "warningMessages": [],
    "errorMessages": [
      {
        "type": "ERROR",
        "code": "XSD_ZATCA_INVALID",
        "category": "XSD validation",
        "message": "Schema validation failed; XML does not comply with UBL 2.1 standards in line with ZATCA specifications",
        "status": "ERROR"
      }
    ],
    "status": "ERROR",
    "reportingStatus": "NOT_REPORTED"
  }
}
```

Example Value | XML Schema Error

```
{
  "validationResults": {
    "infoMessages": [],
    "warningMessages": [],
    "errorMessages": [
      {
        "type": "ERROR",
        "code": "XSD_ZATCA_INVALID",
        "category": "XSD validation",
        "message": "Schema validation failed; XML does not comply with UBL 2.1 standards in line with ZATCA specifications. ERROR: org.xml.sax.SAXParseException; lineNumber: 204; columnNumber: 19; cvc-complex-type.2.4.b: The content of element 'cac:Price' is not complete. One of '{\r\nurn:oasis:names:specification:ubl:schema:xsd:CommonBasicComponents-2\":PriceAmount}' is expected.",
        "status": "ERROR"
      }
    ],
    "status": "ERROR",
    "reportingStatus": "NOT_REPORTED"
  }
}
```

Example Value | EN Rules Violation

```
{
  "validationResults": {
    "infoMessages": [],
    "warningMessages": [],
    "errorMessages": [
      {
        "type": "ERROR",
        "code": "BR-26",
        "category": "EN_16931",
        "message": "Each Invoice line (BG-25) shall contain the Item net price (BT-146).",
        "status": "ERROR"
      }
    ],
    "status": "ERROR",
    "reportingStatus": "NOT_REPORTED"
  }
}
```

Example Value | KSA Rules Violation

```
{
  "validationResults": {
    "infoMessages": [
      {
        "type": "INFO",
        "code": "XSD_ZATCA_VALID",
        "category": "XSD validation",
        "message": "Complied with UBL 2.1 standards in line with ZATCA specifications",
        "status": "PASS"
      }
    ],
    "warningMessages": [],
    "errorMessages": [
      {
        "type": "ERROR",
        "code": "BR-KSA-06",
        "category": "KSA",
        "message": "The invoice transaction code (KSA-2) must exist and respect the following structure: NNPNESB where NN (positions 1 and 2) = invoice subtype: - 01 for tax invoice - 02 for simplified tax invoice P (position 3) = 3rd Party invoice transaction, 0 for false, 1 for true N (position 4) = Nominal invoice transaction, 0 for false, 1 for true E (position 5) = Exports invoice transaction, 0 for false, 1 for true S (position 6) = Summary invoice transaction, 0 for false, 1 for true B (position 7) = Self billed invoice",
        "status": "ERROR"
      }
    ],
    "status": "ERROR",
    "reportingStatus": "NOT_REPORTED"
  }
}
```

Example Value | Invalid QR Code

```
{
  "validationResults": {
    "infoMessages": [
      {
        "type": "INFO",
        "code": "XSD_ZATCA_VALID",
        "category": "XSD validation",
        "message": "Complied with UBL 2.1 standards in line with ZATCA specifications",
        "status": "PASS"
      }
    ],
    "warningMessages": [],
    "errorMessages": [
      {
        "type": "ERROR",
        "code": "QR_CODE_INVALID",
        "category": "QR_CODE_VALIDATION",
        "message": "Failed to validate QR code",
        "status": "ERROR"
      }
    ],
    "status": "ERROR",
    "reportingStatus": "NOT_REPORTED"
  }
}
```

Example Value | Missing QR Code

```
{
  "validationResults": {
    "infoMessages": [
      {
        "type": "INFO",
        "code": "XSD_ZATCA_VALID",
        "category": "XSD validation",
        "message": "Complied with UBL 2.1 standards in line with ZATCA specifications",
        "status": "PASS"
      }
    ],
    "warningMessages": [],
    "errorMessages": [
      {
        "type": "ERROR",
        "code": "QR_CODE_INVALID",
        "category": "QR_CODE_VALIDATION",
        "message": "Invalid QR code format, Please follow the ZATCA QR code specifications",
        "status": "ERROR"
      }
    ],
    "status": "ERROR",
    "reportingStatus": "NOT_REPORTED"
  }
}
```

Example Value | Invalid Signature

```
{
  "validationResults": {
    "infoMessages": [
      {
        "type": "INFO",
        "code": "XSD_ZATCA_VALID",
        "category": "XSD validation",
        "message": "Complied with UBL 2.1 standards in line with ZATCA specifications",
        "status": "PASS"
      }
    ],
    "warningMessages": [],
    "errorMessages": [
      {
        "type": "ERROR",
        "code": "invalid-digital-signature",
        "category": "SIGNATURE_ERRORS",
        "message": "Invalid digital signature",
        "status": "ERROR"
      }
    ],
    "status": "ERROR",
    "reportingStatus": "NOT_REPORTED"
  }
}
```

401 Returned when username and password are not added or added as wrong values.

Media type

application/json

Example Value | Unauthorized

```
{
  "timestamp": 1654514661409,
  "status": 401,
  "error": "Unauthorized",
  "message": ""
}
```

Code	Description	Links
406	Returned when accept version header is anything other than V2	
	Media type	
	text/plain;charset=UTF-8	
	Example Value	Not Acceptable
	"This Version is not supported or not provided in the header."	
409	Invoice was already Reported successfully earlier.	
	Media type	
	application/json	
	Example Value	Invoice was already Reported successfully earlier
	{"validationResults":{"infoMessages":[],"warningMessages":[],"errorMessages":[{"type":"ERROR","code":null,"category":null,"message":"Invoice was already Reported successfully earlier.","status":"ERROR"}],"status":"ERROR"},"reportingStatus":"NOT_REPORTED"}	
500	HTTP Internal Server Error. Returned when the service faces internal errors.	
	Media type	
	application/json	
	Example Value	Internal Server Error
	{"category":"HTTP-Errors","code":"500","message":"Something went wrong and caused an Internal Server Error."}	

Schemas	
InfoModel { description: An object representing the result of the clearance or reporting API endpoints when the clearance flag is turned on or off. Basically, it shows an informational message instructing the client to see the other api. message string }	
ErrorModel { description: An object representing the structure of the error object returned by the API endpoints. Specifically, it includes the Category of the error, its code and message. category string code string message string }	
WarningModel { description: An object representing the structure of the warning object returned by the API endpoints. Specifically, it includes the Category of the warning, its code and message. category string code string message string }	
validationResultsModel { description: An object representing the structure of the validation results returned by the API endpoints. Specifically, it includes the invoice hash, status, and lists of info, warning, and error messages. infoMessages array {"\$ref":"#/components/schemas/InfoModel"} warningMessages array {"\$ref":"#/components/schemas/WarningModel"} erroMessages array {"\$ref":"#/components/schemas/ErrorModel"} status string ["PASS","WARNING","ERROR"] }	
InvoiceResultModel { description: An Object the represents the response of the API endpoint where it shows the results including status, warnings (if any), and error (if any) validationResults object {"\$ref":"#/components/schemas/validationResultsModel"} reportingStatus string ["REPORTED","NOT_REPORTED"] }	
InvoiceRequest { description: An object representing the structure of the clearance endpoint request. Specifically, it has the the submitted document hash and the base64 representation of the invoice. invoiceHash string invoice string }	